CLAUDE CERTIFIED ARCHITECT — PROFESSIONAL

Objective 36: Configure Claude Tools and Environments for Teams

Exam objective: Configure Claude Code and related tooling across organisation, project, and user levels — including managed settings, permissions, MCP servers, hooks, and shared project instructions — to support consistent, secure team environments.

Q1 **A mid-size fintech company wants every engineer's Claude Code installation to enforce a deny-list on `rm -rf` style commands regardless of what individual developers configure locally. Which mechanism should the platform team use?**

A. A project-level `.claude/settings.json` committed to each repo

B. A user-level settings file in each engineer's home directory

C. **Managed settings deployed by the organisation that take precedence over all other settings**

D. A CLAUDE.md instruction telling Claude not to run destructive commands

CORRECT ANSWER C

WHY Managed settings are enforced organisation-wide and override project, local, and user settings, making them the only reliable way to guarantee a non-negotiable policy. Project and user settings can be edited by individual developers, and CLAUDE.md is advisory instruction text, not an enforced permission control.

Q2 **A platform engineer is designing the settings hierarchy for a new onboarding program. They want project-specific defaults that apply to everyone cloning the repo, but still allow individual developers to override non-critical preferences locally. What is the correct layering approach?**

A Put everything in managed settings so nothing can be overridden

B Use project-level settings checked into the repo for shared defaults, and let user/local settings override non-critical preferences beneath managed policy

C Put all configuration into CLAUDE.md since it is read first

D Avoid project settings entirely and rely only on environment variables

CORRECT ANSWER B

WHY The settings hierarchy is designed exactly for this: managed settings set hard limits, project settings establish shared team defaults, and user/local settings allow personal customization within those bounds. CLAUDE.md conveys behavioral instructions, not structured permission/tooling configuration, and skipping project settings would lose the shared-defaults benefit.

Q3 **A healthcare software team must ensure Claude Code never has standing access to read files containing patient records outside of an explicitly reviewed directory. Which configuration best achieves this?**

A A CLAUDE.md note asking Claude to avoid PHI directories

B A permission rule that denies file access outside the approved directory, enforced via managed or project settings

C A hook that runs after every file read to check for PHI

D Relying on the model's training to refuse sensitive file access

CORRECT ANSWER B

WHY Explicit deny permission rules scoped to directories are the deterministic, enforceable control for restricting file access; they can't be socially-engineered away like a CLAUDE.md suggestion. A post-hoc hook after the read occurs is too late, and relying on model judgment alone is not a reliable security boundary.

Q4 **An engineering org runs Claude Code across 40 repositories. They want a single external ticketing integration (e.g., Jira) available to every team without each repo maintainer having to configure it separately. What should they do?**

A Configure the MCP server for the ticketing integration in each repository's local settings individually

B **Define the MCP server configuration at the organisation (managed) level so it is distributed to all teams automatically**

C Ask each developer to install the MCP server manually on their laptop

D Embed ticketing credentials directly into CLAUDE.md

CORRECT ANSWER **B**

WHY Defining shared MCP server configuration at the managed/organisational level ensures consistent distribution without per-repo duplication effort, which is the scalable approach for 40 repositories. Manual per-repo or per-developer configuration doesn't scale and embedding credentials in CLAUDE.md is both insecure and not how MCP server config works.

Q5 **A new hire asks why Claude Code in their current project behaves differently than in a personal side project — specific tools are blocked that worked fine before. What is the most likely explanation?**

A Claude Code has a random variance in tool availability

B The work project is governed by managed settings and/or project-level permission rules that restrict tools beyond the user's personal defaults

C Claude Code caches old settings indefinitely

D The side project uses a newer version of Claude Code

CORRECT ANSWER **B**

WHY Differences in tool behavior across projects are almost always due to layered settings — managed policy or project-level permission rules apply additional restrictions not present in a personal, unmanaged project. There is no random variance, and version differences wouldn't typically explain specific tool blocking tied to project context.

Q6 **A team lead wants all commits made with Claude Code assistance to automatically run a secret-scanning check before any file write is finalized, and wants this enforced deterministically rather than hoping Claude remembers. What should they implement?**

A A reminder in CLAUDE.md to scan for secrets before writing

B A pre-tool-use hook that runs a secret scanner and blocks the write if secrets are detected

C A permission rule set to "ask" for all file writes

D Training developers to manually review every diff

CORRECT ANSWER **B**

WHY Hooks provide deterministic, code-enforced lifecycle control — a pre-tool-use hook can programmatically block an action based on scan results, which is far more reliable than a text instruction or manual review. "Ask" permissions add a prompt but don't run an automated check, and CLAUDE.md is non-binding guidance.

Q7 An architecture team is defining shared project instructions so that every engineer working in a monorepo gets the same context about coding conventions, directory structure, and testing commands. Where should this information live?

A. In each engineer's personal user settings

B. In a CLAUDE.md file at the project root (and nested CLAUDE.md files in subdirectories as needed)

C. In the managed settings JSON file

D. In a Slack pinned message

CORRECT ANSWER B

WHY CLAUDE.md is specifically designed to hold persistent, shared project instructions and context that Claude Code reads automatically, including nested files for subdirectory-specific conventions. Managed settings control policy/permissions rather than narrative project context, and user settings or Slack messages wouldn't be consistently available to Claude itself.

Q8 **A security review flags that several engineers have been manually approving "allow" prompts for a risky MCP tool without understanding the implications. What governance change best addresses this at scale?**

A Remove all permission prompts entirely so nothing is ever blocked

B **Set the tool's permission explicitly to "deny" or "ask" with required justification at the managed settings level, rather than leaving it to ad hoc per-user allow decisions**

C Send a company-wide email asking people to be careful

D Disable Claude Code for the affected teams permanently

CORRECT ANSWER B

WHY Codifying the permission as deny (or ask with enforced review) at the managed level removes inconsistent ad hoc judgment calls and enforces a consistent security posture across the organisation. Removing prompts entirely increases risk, and an email or blanket disablement neither scales nor solves the underlying configuration gap.

Q9 **A platform team is rolling out Claude Code to 200 developers and wants to standardize which MCP servers are available, which permissions apply by default, and which hooks run — while still letting individual repos add project-specific MCP servers for their own tooling. What is the best structural approach?**

A Put every possible MCP server and permission into managed settings so no project can add anything

B **Set baseline managed settings for org-wide policy/standard MCP servers, and allow project-level settings to extend with additional MCP servers within the bounds managed settings permit**

C Give every developer admin rights to configure their own MCP servers from scratch

D Avoid managed settings altogether to prevent complexity

CORRECT ANSWER **B**

WHY This layered approach matches the settings hierarchy's intent — managed settings enforce the non-negotiable org baseline while project settings add scoped, team-specific extensions. Locking everything at the managed level prevents legitimate project flexibility, while no managed settings or unrestricted developer admin rights reintroduce the inconsistency and risk the rollout is meant to solve.

Q10 **A compliance officer at an insurance company asks how the organisation can prove that destructive bash commands are blocked for all Claude Code users, including contractors using their own laptops. Which configuration detail matters most for this guarantee?**

A Whether CLAUDE.md mentions the policy

B **Whether the permission/deny rule is defined in managed settings, which cannot be overridden by local, user, or project settings**

C Whether the contractors were verbally briefed

D Whether the project has a README

CORRECT ANSWER **B**

WHY Only managed settings sit at the top of the settings hierarchy and cannot be overridden by lower-precedence settings, which is what gives compliance an auditable, provable guarantee regardless of device or user. Verbal briefings, README files, and CLAUDE.md notes are not enforced controls.

Q11 **A team wants Claude Code to automatically format code with Prettier after every file edit, without relying on the model remembering to run it. What is the appropriate configuration?**

A A CLAUDE.md instruction asking Claude to always run Prettier

B **A post-tool-use hook configured to run Prettier automatically after file edit operations**

C A permission rule set to "ask" on file edits

D Manually running Prettier after each Claude session ends

CORRECT ANSWER **B**

WHY Hooks are built for exactly this use case — deterministically triggering an action (like running a formatter) after a specific tool event, independent of whether the model "remembers." A CLAUDE.md instruction is non-binding, an "ask" permission just adds a prompt, and manual runs don't scale across a team.

Q12 **A startup's two developers share a single repository and want lightweight configuration without the complexity of organisation-wide managed settings. What is the most appropriate starting point?**

A Immediately set up managed settings infrastructure before writing any code

B Use project-level settings and a CLAUDE.md file in the repo to establish shared conventions and permissions appropriate to their scale

C Require every commit to go through a compliance review board

D Disable all permissions checks to move faster

CORRECT ANSWER **B**

WHY For a small team, project-level settings plus CLAUDE.md provide sufficient shared configuration and context without the overhead of enterprise-wide managed settings infrastructure, which is appropriate for larger organisations. Disabling permission checks entirely removes useful safety guardrails even at small scale.

Q13 **An enterprise wants to allow read-only MCP integrations (e.g., a documentation search server) to be auto-approved for all users, but require explicit human approval for any MCP server capable of writing to external systems. How should permissions be structured?**

A Set all MCP tools to "allow" for simplicity

B Configure "allow" rules for the read-only MCP tools and "ask" (or "deny" pending review) rules for write-capable MCP tools

C Set all MCP tools to "deny" so nothing works

D Leave permissions unconfigured and let each user decide per session

CORRECT ANSWER **B**

WHY Differentiating permission rules by risk level — allowing safe read-only tools while gating write-capable tools behind explicit approval — reflects proper least-privilege permission design. Blanket allow or deny ignores risk differences, and leaving it unconfigured produces inconsistent, ungoverned behavior across users.

Q14 **A global enterprise has regional subsidiaries that need slightly different compliance rules (e.g., different data residency restrictions) layered on top of a common corporate baseline. How can managed settings support this?**

A Managed settings can only be defined once globally with no regional variation possible

B **Managed settings can be scoped/deployed per organisational unit or environment, allowing a common baseline plus region-specific policy layers**

C Each region must maintain a completely separate, unrelated Claude Code deployment

D Regional differences should be handled entirely through CLAUDE.md text

CORRECT ANSWER **B**

WHY Managed settings deployment can be targeted to different organisational scopes, letting a company maintain a shared baseline while layering region-specific policy on top — this is the intended design for enterprise rollout at scale. Treating CLAUDE.md as the mechanism for binding compliance rules is inappropriate since it's advisory, not enforced.

Q15 A DevOps lead is auditing why a particular engineer's Claude Code session has broader file-write permissions than the rest of the team despite the same managed settings being deployed. What should they check first?

A Whether the engineer has local or user-level settings that are incorrectly permitted to loosen restrictions the managed settings did not explicitly lock down

B Whether the engineer is using a different programming language

C Whether Claude Code was reinstalled recently

D Whether the engineer's monitor resolution differs

CORRECT ANSWER A

WHY If managed settings don't explicitly lock a permission, lower-precedence local or user settings can still broaden it, so auditing those layers is the correct first step to explain unexpected permission differences. Unrelated factors like programming language, reinstallation, or hardware have no bearing on the permission hierarchy.

Q16 **A platform team sets an organisation-wide managed-settings deny rule blocking `Bash(curl:*)` to unapproved domains. A developer's personal user-level settings separately contain an "allow" rule for the same curl pattern because they find the prompts annoying. When the developer runs a matching curl command, what is the outcome?**

A The allow rule wins because user settings are evaluated last and "last write wins"

B The deny rule always wins — permission evaluation resolves deny before ask before allow, and a lower-precedence allow rule cannot carve out an exception to a higher-precedence deny

C Claude Code will randomly choose between the two rules

D The command runs but logs a warning, since deny rules are advisory only

CORRECT ANSWER **B**

WHY Permission rules are evaluated in a fixed deny → ask → allow order, and this evaluation order combines with settings precedence so that a deny rule — especially one set at a higher-precedence layer like managed settings — cannot be overridden by an allow rule at a lower-precedence layer such as user settings. Deny rules are enforced, not advisory, and there is no "last write wins" or random behavior in permission resolution.

Q17 A regulated enterprise wants to let developers install Claude Code plugins (bundles of skills, commands, subagents, hooks, and MCP servers) only from its internally vetted plugin marketplace, while blocking installation from any public or unknown third-party marketplace. Which managed-settings capability should they configure?

A. A CLAUDE.md note listing the approved marketplace

B. Managed-settings marketplace controls such as `strictKnownMarketplaces` and `blockedMarketplaces` that restrict which plugin marketplaces are reachable at all

C. Asking developers to self-certify that they only installed approved plugins

D. Disabling the Skills feature entirely so no plugins can be installed

CORRECT ANSWER B

WHY Managed settings expose marketplace allow/deny controls (e.g., restricting to known marketplaces and blocking others) so an organisation can govern plugin distribution deterministically, which matters because plugins can bundle powerful capabilities like hooks and MCP servers. A CLAUDE.md note or self-certification is non-enforced guidance, and disabling Skills entirely sacrifices legitimate productivity capability instead of scoping the actual risk (untrusted marketplaces).

Q18 A financial services firm requires that if an engineer's laptop cannot reach the managed-settings distribution server at the start of a session (for example, VPN is down), Claude Code must refuse to start rather than silently fall back to a stale cached policy that might be missing a recently added restriction. Which setting enforces this "fail closed" behaviour?

A `forceRemoteSettingsRefresh`, which blocks session start if current organisational policy cannot be fetched

B A longer session timeout value

C A CLAUDE.md instruction asking Claude to check for connectivity first

D Setting the default permission mode to "ask" for all tools

CORRECT ANSWER A

WHY `forceRemoteSettingsRefresh` is designed specifically for this fail-closed guarantee — if the latest managed policy cannot be confirmed from the server, the session refuses to start rather than operating under potentially outdated cached rules. Session timeouts, CLAUDE.md notes, and permission-mode defaults don't address the specific risk of stale policy being silently applied when connectivity is lost. --